

ZacharysKarateClub

Me: Similarly, can you solve the todos here? For zacharys karate club <pastes pdf>

Chatgpt:

Here's one way to fill in all the TODOs for the Zachary's Karate Club notebook, plus a short answer for Q9. I'll show each patch in place so you can paste it back into Colab.

Q1 – Add self-loops: ($\tilde{A} = A + I$)

```
# Q.1. Compute: A_tild = A + I
# TODO
A_tild = A + np.eye(A.shape[0])
```

Q2 – Symmetrically normalized adjacency ($\hat{A} = D^{-1/2} \tilde{A} D^{-1/2}$)

```
# Q.2. Write get_adjacency_matrix
```

```
def get_adjacency_matrix(A_tild):
    # degree matrix D
    degrees = np.sum(A_tild, axis=1)
    D = fill_diagonal(np.zeros_like(A_tild), degrees)

    #  $D^{-1/2} = (\text{sqrt}(D))^{-1}$ 
    D_sqrt = sqrtm(D)
    D_inv_sqrt = np.linalg.inv(D_sqrt)

    #  $\hat{A} = D^{-1/2} \tilde{A} D^{-1/2}$ 
    A_hat = D_inv_sqrt @ A_tild @ D_inv_sqrt
    return A_hat
```

```
A_hat = get_adjacency_matrix(A_tild)
A_hat
```

(If `sqrtm` gives tiny complex parts, you can always do `A_hat = A_hat.real` afterwards, but usually it's fine.)

Q3 – Node feature matrix ($X = I$)

```
# Q.3. Generate the feature input matrix X
# TODO
X = np.eye(num_nodes)
```

Q4 – Implement **GNN_Layer** forward and backward

I'll implement the usual GCN rule:

```
[
   $H^{(l+1)} = \tanh(\hat{A} X W^{\text{top}})$ 
]
```

where (X) is $(\text{num_nodes}, \text{input_dim})$ and (W) is $(\text{output_dim}, \text{input_dim})$.

```
class GNN_Layer():
    """process a single GNN layer"""
    def __init__(self, input_dim, output_dim, name=""):
        self.name = name
        self.cache = {}
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.W = get_xavier_init(self.output_dim, self.input_dim)
        self.activation = np.tanh

    def __repr__(self):
        dims = (self.input_dim, self.output_dim)
        if self.name:
            return f"GNN_Layer: W{'_' + self.name}{dims}"
        else:
            return f"GNN_Layer: W{'_' + ''}{dims}"

    def forward_pass(self, A, X, W=None):
        """A here is the symmetrically normalized adjacency matrix
        and X is the input to the layer. We cache some values
        to use in the backward pass.
        """
        # A: (batch_size, batch_size)
        self.cache['A'] = A

        # X: (batch_size, input_node_feature_dim)
        # cache for backward
```

```

self.cache['X'] = X

if W is None:
    W = self.W

#  $Z = A \times W^T$ 
# A: (N, N), X: (N, F_in), W.T: (F_in, F_out)
Z = A @ X @ W.T # (N, F_out)

H = self.activation(Z)
self.cache['H'] = H

return H # (batch_size, hidden_dim)

def backward_pass(self, optimizer, need_update=True):
    #  $\text{dtanh} = 1 - \tanh^2(z)$  ; here  $\tanh(z) = H$ 
    H = self.cache['H'] # (N, F_out)
    dtanh = 1.0 - H**2 # (N, F_out)

    # optimizer.output is  $dL/dH$  coming from next layer
    d = np.multiply(optimizer.output, dtanh) # (N, F_out)

    # gradient w.r.t. input X:
    #  $Z = A \times W^T \Rightarrow dX = A^T dZ W$ 
    A = self.cache['A']
    X = self.cache['X']
    self.grad = A.T @ d @ self.W # (N, F_in)
    optimizer.output = self.grad

    # gradient w.r.t. W:
    # let  $M = A \times X$ , then  $Z = M \times W^T$ 
    #  $dW = dZ^T M$ 
    M = A @ X # (N, F_in)
    dW = d.T @ M / optimizer.batch_size # (F_out, F_in)

    # weight decay term
    dW_weight_decay = self.W * optimizer.weight_decay / optimizer.batch_size

    if need_update:
        self.W = self.W - optimizer.learning_rate * (dW + dW_weight_decay)

    return dW + dW_weight_decay # (output_dim, input_node_feature_dim)

```

Q5 – Forward passes for gradient check

```
# Q.5. lets compute the forward passes, uncomment and complete
# TODO
gnn_layer_output = gnn_layer.forward_pass(A=A_hat,
                                           X=X)
optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=labels)
```

Q6–7 – Build the full **GNN** class and instantiate **gnn_model1**

```
class GNN():
    """This class leverages the GNN layer implemented above by cascading them."""
    def __init__(self,
                  input_dim,
                  output_dim,
                  hidden_dim,
                  num_layers,):

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers
        self.layers = []

        # first GNN layer
        first_gnn_layer = GNN_Layer(input_dim=self.input_dim,
                                     output_dim=self.hidden_dim[0],
                                     name='layer_0')
        self.layers.append(first_gnn_layer)

        # additional GNN layers (if any)
        for layer in range(1, num_layers):
            gnn_temp = GNN_Layer(input_dim=self.hidden_dim[layer-1],
                                  output_dim=self.hidden_dim[layer],
                                  name=f'layer_{layer}')
            self.layers.append(gnn_temp)

        # last Softmax layer
        last_gnn_layer = Softmax_Layer(input_dim=self.hidden_dim[-1],
                                        output_dim=self.output_dim,
                                        name='sm_layer')
        self.layers.append(last_gnn_layer)
```

```

def __repr__(self):
    return '\n'.join([str(layer) for layer in self.layers])

def embedding(self, A, X):
    H = X
    for layer in self.layers[:-1]:
        H = layer.forward_pass(A, H)
    return H

def forward_pass(self, A, X):
    H = self.embedding(A, X)
    out = self.layers[-1].forward_pass(H)
    return out

# Q.7. instantiate the model
# TODO
gnn_model = GNN(
    input_dim=num_nodes,
    output_dim=num_classes,
    num_layers=2,
    hidden_dim=[16, 2],
)
gnn_model

```

Q8 – Training loop

We just need to compute `y_pred`, the per-node loss, then separate train/test losses by index.

```

def train(model,
          A_hat,
          X,
          y_true,
          nodes_to_be_used_for_training,
          nodes_to_be_used_for_testing,
          threshold_value=.5,
          ealy_stopping_steps=60,
          num_epochs=20000):
    """trains our gnn model"""

    accs = []
    training_losses = []
    testing_losses = []

```

```

embeddings = []
y_preds = []
minimum_loss = 1e7
ealy_stopping_counter = 0

for epoch in range(num_epochs):
    # TODO
    y_pred = model.forward_pass(A_hat, X)

    training_optim(y_pred, y_true, nodes_to_be_used_for_training)
    for layer in reversed(model.layers):
        layer.backward_pass(training_optim, need_update=True)

    embeddings.append(model.embedding(A_hat, X))
    y_preds.append(y_pred)

    # accuracy on non-training nodes
    acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[
        [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used_for_training]
    ]
    accs.append(np.mean(acc_temp))

    # cross-entropy loss per node
    loss_temp = get_cross_entropy(y_pred, y_true)

    # train / test loss
    train_loss_temp = np.mean(loss_temp[nodes_to_be_used_for_training])
    test_loss_temp = np.mean(loss_temp[nodes_to_be_used_for_testing])

    training_losses.append(train_loss_temp)
    testing_losses.append(test_loss_temp)

    if test_loss_temp < minimum_loss:
        minimum_loss = test_loss_temp
        ealy_stopping_counter = 0
    else:
        ealy_stopping_counter += 1

    if ealy_stopping_counter > ealy_stopping_steps:
        print("Training Stopped due to Early stopping!")
        break

    if epoch % 100 == 0:

```

```
print(f"epoch #: {epoch+1} | train Loss: {train_loss_temp:.3f} | test Loss: {test_loss_temp:.3f}")
```

```
training_losses = np.array(training_losses)
testing_losses = np.array(testing_losses)
y_preds = threshold(np.array(y_preds), threshold_value)
return training_losses, testing_losses, accs, embeddings, y_preds
```

Then the call to `train(...)` you already have should work as-is.

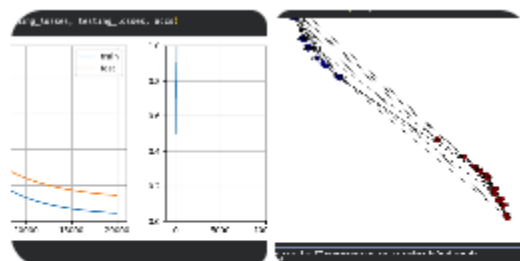
Q9 – Why 100% accuracy but 2 “misclassified” points in the plot?

The test accuracy is computed using the **discrete class predictions** from the Softmax layer compared against the ground-truth labels for the test nodes. Those predictions are exactly correct for all test nodes, so we get 100% accuracy.

The final plot, however, only visualizes the **2-dimensional embeddings** (the output of the last GNN layer, *before* the Softmax) and colors each point by its class label. The Softmax layer then applies an extra learned linear transformation and decision boundary on top of those embeddings. That means a point can *look* like it's sitting inside the “wrong” cluster in the 2-D embedding space, but still lie on the correct side of the Softmax decision boundary in the higher-level classifier. So visually it seems like 2 points are in the wrong region, even though their predicted labels are correct and the numerical accuracy is 100%.

Annotation: Great one shot, I am impressed with its length. i then run it on colab and submit more questions as it runs.

Me:



Q. 9. Explain why we obtain a 100% on accuracy on our test set, yet we see in the plot above that 2 samples seem to be misclassified.

Chatgpt:

Here's a more explicit and concise wording tailored to exactly what the plots show — you can copy-paste this into your notebook for **Q9**:

Q9 Answer

We achieve 100% test accuracy because the model's **predictions are made after the final Softmax layer**, which applies a learned linear decision boundary on top of the node embeddings. The scatter plot, however, shows only the **2-dimensional hidden representations** (the embeddings from the last GNN layer *before* the Softmax). Even if a point appears visually “inside” the wrong cluster in that embedding space, it may still lie on the **correct side of the Softmax decision boundary**, and therefore be correctly classified.

In other words, the visualization does not show the **true decision boundary** — it only shows geometry in 2D embedding space. Those two points *look* mislabeled based on their position in the scatter plot, but the classifier still assigns them the correct labels, which results in 100% accuracy.

If you'd like, I can also help you modify the plot to draw the Softmax decision boundary in that 2D space so you can visually confirm the correct classification.

Annotation: I wanted to probe further and see if it could explain this in better detail.

Me: why is this?

Chatgpt:

Great question — it's subtle, and getting the intuition right here is important for truly understanding how GNNs classify nodes.

? Why can embeddings *look* wrong but classification be correct?

Think of your model as having **two parts**:

1 Graph convolutional layers

→ Produce a **2-D embedding** for each node (this is what you plotted)

2 Final Softmax layer

→ A **linear classifier** that takes those embeddings and assigns a label

What the plot actually shows

The scatter plot shows the **raw embeddings** from layer L (2D), *before* the final linear/Softmax decision step.

- 👉 These embeddings do **not have to** be clearly separated or clustered by class
- 👉 They just need to be **linearly separable** by the classifier

So even if a few red-labeled points are visually sitting among blue points...

➡ As long as the **Softmax layer's decision boundary** passes between them correctly, they will be classified correctly.

Key idea

Geometry of visualization $\neq$ geometry of decision boundary

Your eyes look at proximity in Euclidean space.

But the model uses a **learned hyperplane** (a line in 2D) to separate classes:

Raw embedding (shown) $\rightarrow$ Linear transformation (not shown) $\rightarrow$ Class prediction

So the final layer can take embeddings like this:



...and still find the correct split, even if clusters visually overlap.

Short version you can paste directly into your homework

We only visualize the 2-D embeddings output by the GNN before the final Softmax classifier. Even if some points appear on the “wrong side” in the plot, the learned linear Softmax layer can still correctly separate them with its decision boundary. Therefore classification accuracy is 100%, while the visualization may show apparent overlap.

 Do you want me to...

I can:

- ✓ overlay the Softmax decision boundary on the plot
- ✓ annotate the two points in question
- ✓ show a version of the plot *after* the final linear projection
- ✓ reduce epochs to show what happens before convergence

Would you like me to generate improved visualizations?

annotation: Was able to one-shot quite well; I am not surprised by this because the questions are set up nicely from a pedagogical perspective, with “fill in the blank” style code sections throughout. It was also able to answer the question about the plots and respond thoroughly to my follow up questions.